

Lecture 9: Intermediate Code Optimization

中文学习笔记

基于课件 8.Code_Optimization.pdf 整理

2026 年 6 月 26 日

目录

阅读导引	3
1 编译阶段与优化入口 [p1–3]	3
2 优化目标、发生位置与基本原则 [p4–6]	4
2.1 优化器做什么 [p4]	4
2.2 优化发生在哪里 [p5]	4
2.3 优化原则 [p6]	4
3 优化分类：布局相关与代码相关 [p7–14]	5
3.1 优化本质是程序变换 [p7–9]	5
3.2 代码布局示例 [p10]	5
3.3 数据布局示例 [p11–12]	6
4 代码相关优化的四类变换 [p15]	6
5 控制流分析、基本块与 CFG [p16–22]	7
5.1 控制流分析 [p16]	7
5.2 基本块 [p17]	7
5.3 控制流图 [p18]	8
5.4 构造 CFG [p19–22]	8
6 局部优化与 DAG [p23–33]	9
6.1 局部优化与全局优化 [p23]	9

目录	2
6.2 公共子表达式消除与死代码删除 [p25]	9
6.3 基本块 DAG [p26-29]	10
6.4 代数恒等式与强度削减 [p30-31]	11
6.5 常量折叠与常量传播 [p32-33]	12
7 全局优化、保守性与数据流分析 [p34-41]	13
7.1 全局优化必须对所有路径正确 [p35-36]	13
7.2 保守性原则 [p37]	13
7.3 数据流分析 [p38]	14
7.4 全局常量传播 [p39-41]	14
8 LLVM Pass 与优化等级 [p42-45]	15
8.1 LLVM Pass [p42-43]	15
8.2 优化等级 [p44-45]	15
9 总结与阅读 [p46-47]	16
10 DAG 构造复习 [p48-50]	16
11 练习题与解答	17
复习建议	19

阅读导引

本讲讲中间代码优化。前一讲已经把带标注 AST 翻译成 IR，本讲关心如何在保持程序语义等价的前提下改写 IR，使程序更快、更小、更省内存或能耗更低。主线是：先理解优化目标和分类，再构造基本块与控制流图，随后在基本块内做局部优化，在 CFG 上做全局优化，最后联系 LLVM Pass 和优化等级。

本讲主线

代码优化不是随意“改代码”，而是在证明语义等价的前提下做变换。局部优化依赖基本块和 DAG，全局优化依赖 CFG 和数据流分析。编译器宁可保守地放弃一次优化，也不能做一次可能改变结果的优化。

1 编译阶段与优化入口 [p1-3]

代码优化位于中间代码生成之后、目标代码生成之前：

Semantic Analyzer → Intermediate Code Generation → Code Optimizer → Code Generator.

优化器的输入通常是中间表示 IR 和符号表，输出仍是 IR，只是这份 IR 在执行时间、空间、能耗或代码大小上更好。

课件第 3 页的 while 示例：

```
while (y < z) {  
    int x = a + b;  
    y += x;  
}
```

中间代码中 $a + b$ 被计算后放入临时变量 $t1$ ，并移到循环外复用：

```
t1 = a + b  
goto L1  
L2:  
t2 := y + t1  
y := t2  
L1:  
if y < z goto L2
```

这体现了循环不变代码外提的思想：若某个表达式在循环中每次结果都相同，就可以在循环前计算一次，避免重复计算。

2 优化目标、发生位置与基本原则 [p4-6]

2.1 优化器做什么 [p4]

课件把优化器分成三类活动：

- 控制流分析 (control-flow analysis)：弄清程序可能按哪些路径执行。
- 数据流分析 (data-flow analysis)：弄清变量值、定义、使用等信息如何沿路径传播。
- 代码变换 (code transformation)：在分析信息支持下修改 IR。

代码优化

代码优化是在保持程序可观察行为不变的前提下，对程序表示做变换，以提升效率和性能。目标可包括减少执行时间、内存使用、能耗、二进制大小或计算复杂度。

2.2 优化发生在哪里 [p5]

位置	例子	特点
人工源码层面	选择更合适算法和数据结构，写出编译器容易优化的代码	程序员负责，收益可能很大，但依赖经验
编译器 IR 层面	生成更高效 TAC/SSA，做 CSE、DCE、常量传播等	本讲重点，机器无关，便于复用
目标代码生成层面	选择指令、分配寄存器、调度指令	依赖目标机器 ISA 和硬件特性
汇编器/链接器层面	链接时优化、重排代码布局	编译后期，能利用跨文件或二进制级信息

2.3 优化原则 [p6]

优化希望改善以下一个或多个指标：

- 执行时间。
- 内存使用。
- 能源消耗。
- 二进制可执行文件大小。

但优化必须遵守三个原则：

- 等价性：优化前后程序结果不能改变。这是最重要的正确性要求。
- 高效性：优化后目标代码应更快或占用更少存储空间。
- 成本效益：优化本身不能太昂贵，否则编译时间过长可能得不偿失。

3 优化分类：布局相关与代码相关 [p7-14]

3.1 优化本质是程序变换 [p7-9]

编译器优化本质上是一类程序变换，包括：

- 删除：删掉不影响结果的代码。
- 添加：插入能提升性能的辅助指令。
- 移动：调整指令顺序或代码布局。
- 修改：把昂贵操作替换成便宜操作。

课件把优化分成两大类：

类别	目标	例子
Layout-related transformations	优化代码和数据在内存中的放置，最大化空间局部性	函数布局、字段顺序调整、AOS转SOA
Code-related transformations	优化生成什么代码，尽量执行更少、更便宜的指令	CSE、DCE、常量折叠、常量传播、强度削减

空间局部性

空间局部性指：访问某个内存位置后，附近位置很可能很快也被访问。缓存一次会装入一整条 cache line，因此若相关代码或数据靠得近，就更容易命中缓存。

3.2 代码布局示例 [p10]

课件假设数据 cache 一条 line 有 N 个 word，每个函数大小为 $N/2$ word，访问序列为：

$$g, f, h, f, h, f, h.$$

若代码布局为 f, g, h ，访问时可能产生 6 次 cache miss；若布局为 f, h, g ，由于 f 和 h 放在同一 cache line 中，访问序列反复在 f/h 间切换时可复用缓存，miss 可降到 2 次。

通俗理解：把经常相邻执行的函数放近，像把经常一起用的书放在同一个书架格子里，拿一次就能顺带拿到另一部分。

3.3 数据布局示例 [p11-12]

示例 1：调整结构体字段顺序。

```
struct S {
    int a;
    int b[10];
    int c;
} obj[100];

for (...) {
    ... = obj[i].a + obj[i].c;
}
```

若把字段顺序改为：

```
struct S {
    int a;
    int c;
    int b[10];
} obj[100];
```

那么循环中一起访问的 a 和 c 更可能落在同一 cache line 中，访问 c 更可能命中缓存。

示例 2：AOS 转 SOA。

```
// AOS: array of structs
struct S { int a; int b; } objs[100];

// SOA: struct of arrays
struct S { int a[100]; int b[100]; } objs;
```

如果两个循环分别只访问所有 a 或所有 b ，SOA 让同字段连续排列，空间局部性更好。

4 代码相关优化的四类变换 [p15]

变换	含义	示例
Modifying	修改指令形式，用更便宜指令替代昂贵指令	$A = 2 * a$ 改为 $A = a << 1$
Deleting	删除不影响结果的指令	$A = 2; A = y;$ 中前一条赋值可删

Moving	调整独立指令顺序，改善并行性或隐藏延迟	$A = x/y; B = A + 1; C = y;$ 改为 $A = x/y; C = y; B = A + 1;$
Inserting	插入有助于性能的指令	链表遍历中提前 <code>prefetch(p->next)</code>

这些变换对应常见优化：

- 强度削减（strength reduction）是 modifying。
- 死代码删除（dead code elimination）是 deleting。
- 代码调度（code scheduling）是 moving。
- 数据预取（data prefetching）是 inserting。

5 控制流分析、基本块与 CFG [p16-22]

5.1 控制流分析 [p16]

Control-Flow Analysis

控制流分析是静态分析程序执行路径的方法。其结果通常表示为控制流图 CFG，用于后续全局分析和优化。

命令式语言中控制流通常由 if、while、for、goto、return 等结构显式决定。要构造 CFG，第一步是把代码划分为基本块。

5.2 基本块 [p17]

Basic Block

基本块是直线代码序列：除第一条指令外没有其他标签或跳转入口；除最后一条指令外没有跳转。也就是说，一个基本块只有一个入口和一个出口。

基本块的性质：

- 只能从块开头进入。
- 只能从块末尾跳出。

- 块内指令要么全部执行，要么都不执行。
- 基本块不能再被进一步切分为更小的控制流单元。
- CFG 中的节点就是基本块。

局部优化只在单个基本块内进行；全局优化跨基本块，因此需要 CFG。

5.3 控制流图 [p18]

Control Flow Graph

控制流图是有向图：每个节点代表一个基本块，每条有向边代表执行流可从一个基本块转移到另一个基本块。

边可能来自：

- 一个基本块结束后自然落入下一个基本块。
- 条件分支导致控制流分叉。
- 多条路径汇合到同一基本块。

CFG 广泛用于表示单个函数，并支撑全局程序分析和优化。

5.4 构造 CFG [p19-22]

第一步：找 leader。满足以下任一条件的指令是 leader：

1. 程序第一条指令。
2. 条件或无条件跳转的目标指令。
3. 紧跟在条件或无条件跳转之后的指令。

从一个 leader 开始，到下一个 leader 之前的所有指令构成一个基本块。

第二步：添加基本块之间的边。若 B_1 和 B_2 满足以下情况，则添加边：

- B_2 紧跟 B_1 ，且 B_1 可能 fall through 到 B_2 。条件跳转的 false 分支和非跳转结束都可能 fall through；无条件跳转不会 fall through。
- B_1 以跳转到 B_2 结束，即使 B_2 不紧跟 B_1 。

课件奇偶判断例子：


```
0: t0 = read_num
1: if t0 mod 2 == 0
2: print t0 + " is even."
3: goto 5
4: print t0 + " is odd."
5: end program
```

leaders 为0,2,4,5: 0 是第一条, 2 和 5 是跳转目标, 4 是紧跟跳转后的指令。

6 局部优化与 DAG [p23-33]

6.1 局部优化与全局优化 [p23]

局部优化

局部优化只在一个基本块内部进行。由于基本块内没有分支, 块内指令要么按顺序执行一次, 要么完全不执行, 因此局部优化最简单。

全局优化

全局优化跨基本块进行, 作用域可包含 if、while、for 等控制流结构。这里的 global 通常指函数内跨基本块, 不一定跨整个程序。

局部优化例子:

- 常量折叠: `const int a = 2 + 3;` 可变为 `a=5`, 再用于后续表达式。
- 公共子表达式消除: 同一个基本块内重复计算的表达式可复用前一次结果。

6.2 公共子表达式消除与死代码删除 [p25]

Common Subexpression Elimination

公共子表达式消除把多次计算同一值的表达式改为只计算一次, 后续使用第一次结果。

课件例子:

```
y = x + z;  
y = x * x + (x / 3);  
z = x * x + y;
```

拆成 TAC 后, $x * x$ 被计算两次:

```
t1 = x * x  
t2 = x / 3  
y = t1 + t2  
t3 = x * x  
z = t3 + y
```

可优化为:

```
t1 = x * x  
t2 = x / 3  
y = t1 + t2  
z = t1 + y
```

Dead Code Elimination

若一条指令的结果之后从未被使用, 则该指令是死代码, 可从指令流中删除。

死代码删除常和其他优化连用: CSE、常量传播之后常会产生新的死代码。

6.3 基本块 DAG [p26-29]

DAG of Basic Block

基本块 DAG 用有向无环图表示基本块内部的值流。叶子节点代表变量初始值, 中间节点代表运算, 节点可附着一个或多个变量名。它能帮助识别公共子表达式和死代码。

构造算法:

1. 为基本块中出现的变量初始值创建叶子节点。
2. 对每条语句创建与该语句关联的节点 N 。

3. N 的子节点是该语句操作数在此语句之前的最近定义。
4. 用语句中的运算符标记 N 。
5. 某些节点被指定为输出节点，表示块出口仍需保留的值。

课件基本块：

- (1) $a = b + c$
- (2) $b = a - d$
- (3) $c = b + c$
- (4) $d = a - d$

可用 DAG 发现：

- 若 b 在块出口不活跃，则语句 (2) 的结果可不用保留。
- 若 b 和 d 都在出口活跃，(2) 与 (4) 都计算 $a - d$ ，可只计算一次，再复制给另一个变量。
- 若只有 a 在出口活跃，则与 b, c, d 最终值相关的节点都可视为死代码删除。

6.4 代数恒等式与强度削减 [p30-31]

普通 CSE 只识别“语法上明显相同”的表达式；但有些表达式虽然写法不同，数学上等价。课件例子：

- (1) $a = b + c$
- (2) $b = b - d$
- (3) $c = c + d$
- (4) $e = b + c$

第 (1) 和第 (4) 的 $b + c$ 在数学上相等，因为 $(b - d) + (c + d) = b + c$ 。普通 DAG 可能错过，需要代数恒等式帮助。

常见代数恒等式：

$$a * 1 \equiv a, \quad a * 0 \equiv 0, \quad b \& \text{true} \equiv b.$$

重结合和交换律：

$$(a + b) + c \equiv a + (b + c), \quad a + b \equiv b + a.$$

Strength Reduction

强度削减是把昂贵操作替换成便宜操作，例如用移位、加法替代乘法或除法。

例子:

```
x = y / 8  -> x = y >> 3
y = y * 8  -> y = y << 3
x^2        -> x * x
2 * x      -> x + x
```

6.5 常量折叠与常量传播 [p32-33]

Constant Folding

常量折叠是在编译期直接计算常量表达式，避免运行时重复计算。

例子:

```
#define LEN 100
x = 2 * LEN;
if (LEN < 0) print("error");
```

可折叠为:

```
x = 200;
if (false) print("error");
```

随后 DCE 可删除永远不会执行的 if。

Constant Propagation

常量传播是在编译期把已知常量值替换到变量使用处。

局部例子:

```
x = 4
y = x * 3
```

可传播并折叠为:

```
x = 4
y = 12
```

常量传播有局部版本 LCP 和全局版本 GCP。GCP 更强，因为它跨基本块，但也更复杂，需要 CFG 和数据流分析。

7 全局优化、保守性与数据流分析 [p34-41]

7.1 全局优化必须对所有路径正确 [p35-36]

全局优化跨越控制流，因此必须考虑所有可能路径。

可优化例子：

```
X = 3;
if (B > 0)
    Y = Z + W;
Y = 0;
A = 2 * X;
```

从 $X = 3$ 到 $A = 2 * X$ 的所有路径上， X 都没有被改写，所以可优化为：

```
A = 2 * 3;
```

不可优化例子：

```
X = 3;
if (B > 0)
    Y = Z + W;
X = Y;
Y = 0;
A = 2 * X;
```

这里 X 可能被 Y 改写，后面的 X 不一定仍为 3，因此不能传播。

7.2 保守性原则 [p37]

Conservative Analysis

保守分析要求：只有当编译器能证明某性质在某程序点沿所有路径都成立时，才允许基于该性质优化。不确定时必须回答 don't know，并放弃优化。

保守性会损失部分优化机会，但保证正确性。编译器优化的底线是不能改变程序语义。

全局常量传播需要同时知道：

- 控制流：从哪里可能到哪里。
- 数据流：变量值是否在路径上被改写。

7.3 数据流分析 [p38]

Dataflow Analysis

数据流分析是在程序每个点计算某种属性值的编译器分析框架。属性值沿 CFG 边传播，语句可改变这些值。为了可行且正确，数据流框架通常使用保守近似。

以 GCP 为例，程序点处的属性可以表示为：

$$\{A = ?, B = 10, C = ?\}.$$

若有语句：

$$A = B + C$$

且分析知道 $B = 10$ ，则可优化为：

$$A = 10 + C$$

7.4 全局常量传播 [p39-41]

GCP 用以下状态描述变量：

状态	含义
$x = *$	未赋值或默认未知初始状态
$x = 1, x = 2, \dots$	已知变量被赋为某个常量
$x = \#$	变量可能来自多个不同值，无法确定为单一常量

算法思想：

1. 所有变量初始状态为*。
2. 沿 CFG 反复传播和更新状态。
3. 若不同路径给出不同常量，则合并为#。
4. 迭代直到状态不再变化，即达到固定点。

5. 固定点后，把确定常量替换到使用点。

课件例子中，常量可传播到 $X + 1$ 和 $2 * X$ 。GCP 之后，还可以继续做 DCE，删除不再有用的赋值。例如 $X = 2$ 若最终被后续 $X = 3$ 覆盖且无使用，就可删除。

8 LLVM Pass 与优化等级 [p42-45]

8.1 LLVM Pass [p42-43]

LLVM Pass

LLVM 中，优化以 Pass 形式实现。一个 Pass 遍历程序的某个部分，收集信息或执行变换。它接收 LLVM IR，并执行分析和/或变换。

Pass 的特点：

- 可用 `opt` 单独运行某个 Pass。
- 可在源代码到二进制代码的编译流程中间执行。
- 多个 Pass 的顺序由 Pass Manager 管理。

这和本讲模型对应：分析 Pass 提供控制流/数据流信息，变换 Pass 用这些信息改写 IR。

8.2 优化等级 [p44-45]

等级	含义
O0	不优化。编译最快，生成代码最利于调试。
O1	介于 O0 和 O2 之间，做一部分低成本优化。
O2	中等优化等级，启用大多数常用优化。
O3	类似 O2，但启用更耗时或可能增大代码体积的优化，目标是更快运行。
Os	类似 O2，但额外优化代码大小。
Oz	比 Os 更激进地减小代码体积。
O4	启用链接时优化；Clang 支持 O4，opt 不支持。

课件还展示了不同优化等级下 GCC/LLVM benchmark 性能比较，说明优化等级会真实影响性能，但效果也依赖具体程序、编译器和硬件。

9 总结与阅读 [p46–47]

课件总结：

- 布局相关优化目标是最大化空间局部性，包括代码布局、声明顺序和数据结构组织。
- 代码相关优化目标是执行最少、最便宜的指令。
- 控制流分析需要基本块和 CFG。
- 局部优化使用基本块 DAG，可做 CSE、DCE、代数恒等式、常量折叠和常量传播。
- 全局优化跨基本块，需要数据流分析，典型例子是全局常量传播。

推荐阅读 Dragon Book：

- 第 8.4、8.5 节：基于 DAG 的基本块优化。
- 第 9.1、9.6.1、9.6.6 节：循环优化基本概念。
- 第 9.2.1–9.2.4 节：数据流方程和 reaching definition analysis。

10 DAG 构造复习 [p48–50]

课件末尾复习 DAG 构造。三类语句：

$$x = y \text{ op } z, \quad x = \text{op } y, \quad x = y.$$

构造规则：

1. 若操作数 y 尚无节点，创建节点 y 。二元操作中若 z 尚无节点，也创建节点 z 。
2. 对二元操作，创建运算节点 OP，左孩子为 y 节点，右孩子为 z 节点。
3. 对一元操作，若已有同操作符、同孩子的节点，可复用；否则创建新节点。
4. 对复制语句 $x = y$ ，让 x 附着到 y 的节点。
5. 每次重新定义 x ，先从旧节点的标识符列表移除 x ，再把 x 附到新节点。

例子：

```
x = A + B
y = x + 3
z = x * y
w = x * y
u = w
a = 4 / 2
b = - a
```


DAG 可发现 z 和 w 都是 $x * y$, 而 $u = w$ 只是复制; $a = 4/2$ 可常量折叠为 $a = 2$, 因此 $b = -a$ 可变为 $b = -2$ 。若只有 z 和 b 在出口活跃, 则优化后的代码可为:

```
x = A + B
y = x + 3
z = x * y
b = -2
```

11 练习题与解答

题 1: 第 3 页 while 示例做了什么优化?

解答: 做了循环不变代码外提和复用。表达式 $a + b$ 在循环体中每次结果相同, 因此先在循环外计算 $t1 = a + b$, 循环中直接使用 $t1$, 避免每轮重复执行加法。

题 2: 优化必须满足哪些原则?

解答: 至少满足等价性、高效性和成本效益。等价性要求优化后程序结果不变; 高效性要求目标代码更快或更省空间; 成本效益要求优化本身开销不能过高。

题 3: AOS 转 SOA 为什么能提升空间局部性?

解答: AOS 中每个对象的多个字段交错排列; 若循环只访问某一字段, 会不断跳过其他字段。SOA 把同一字段连续放在数组中, 访问该字段时更可能连续命中 cache line。

题 4: 找出如下代码的 leaders 并划分基本块。

```
0: t0 = read_num
1: if t0 mod 2 == 0 goto 2
2: print even
3: goto 5
4: print odd
5: end
```

解答: leaders 为 0、2、4、5。0 是第一条指令; 2 和 5 是跳转目标; 4 是紧跟跳转之后的指令。基本块为 $[0, 1]$ 、 $[2, 3]$ 、 $[4]$ 、 $[5]$ 。

题 5: 局部优化和全局优化的区别是什么?

解答: 局部优化只在一个基本块内进行, 不考虑控制流; 基本块内指令顺序执行, 分析简单。全局优化跨基本块进行, 需要考虑 if/while/for 等控制流路径, 通常依赖 CFG 和数据流分析。

题 6：对如下代码做公共子表达式消除。

```
t1 = x * x
t2 = x / 3
y = t1 + t2
t3 = x * x
z = t3 + y
```

解答： $t1$ 和 $t3$ 都计算 $x * x$ ，且中间没有改变 x 。可删掉第二次计算并改用 $t1$ ：

```
t1 = x * x
t2 = x / 3
y = t1 + t2
z = t1 + y
```

题 7：常量折叠与常量传播有什么区别？

解答： 常量折叠是在编译期计算常量表达式，例如 $2 * 100$ 直接变为 200。常量传播是把已知常量值替换到变量使用处，例如 $x = 4; y = x * 3$ 先变为 $y = 4 * 3$ ，再可折叠为 $y = 12$ 。

题 8：为什么全局优化要保守？

解答： 全局优化跨控制流路径。若某性质只在部分路径成立，基于它做优化会改变程序结果。因此编译器必须证明该性质沿所有路径成立；若不能证明，就回答 don't know 并放弃优化。

题 9：解释 GCP 中 $x = *$ 、 $x = 3$ 、 $x = \#$ 的含义。

解答： $x = *$ 表示未赋值或默认初始状态； $x = 3$ 表示当前能确定 x 为常量 3； $x = \#$ 表示 x 可能来自多个不同值，不能确定为单一常量，因此不能替换。

题 10：对如下代码说明是否可把最后的 X 替换为 3。

```
X = 3;
if (B > 0)
    Y = Z + W;
Y = 0;
A = 2 * X;
```

解答： 可以。所有从 $X = 3$ 到 $A = 2 * X$ 的路径上， X 都没有被重新定义，因此最后的 X 一定为 3。

题 11：下面代码能否把最后的 X 替换为 3？

```

X = 3;
if (B > 0)
    Y = Z + W;
X = Y;
Y = 0;
A = 2 * X;

```

解答：不能。中间有 $X = Y$ ，会重新定义 X ，而 Y 不一定为 3，所以不能安全传播常量 3。

题 12：对 DAG Revisit 示例做优化，假设只有 z 和 b 在出口活跃。

原代码：

```

x = A + B
y = x + 3
z = x * y
w = x * y
u = w
a = 4 / 2
b = - a

```

解答： w 与 z 是同一公共子表达式 $x * y$ ， $u = w$ 在出口不活跃可删除； $a = 4/2$ 可折叠为 $a = 2$ ，若 a 不活跃，则直接生成 $b = -2$ 。优化后：

```

x = A + B
y = x + 3
z = x * y
b = -2

```

复习建议

复习本讲建议按三层掌握：

- 概念层：优化目标、等价性、空间局部性、基本块、CFG、DAG、数据流分析、保守性。
- 局部层：能用 DAG 识别 CSE、DCE、常量折叠、常量传播和代数恒等式。
- 全局层：能解释为什么全局优化需要 CFG 和数据流分析，能判断某个常量是否沿所有路径保持不变。

最容易混淆的是“能看到某条路径上为常量”和“所有路径上都为常量”。全局优化只允许后者。